Técnicas de Inteligência Artificial

PL

Licenciatura em Engenharia e Gestão de Sistemas de Informação

Filipe Portela

Aulas PL

Objetivos

- Aprender a raciocinar e pensar em Inteligência Artificial
- Estruturar o pensamento e modelar “algoritmos” através da linguagem Prolog
- Realizar exercícios práticos
- Aprender metodologias, ferramentas e técnicas de Inteligência Artificial (IA)
- Elaborar os projetos práticos da Unidade Curricular (UC)

Aulas PL

Plataforma de Comunicação

- Todos os alunos devem aceder à plataforma do ioChat e registar nas salas respetivas.
- Devem utilizar as contas já criadas para a disciplina de Programação Web e entrar em
  - TIA

Aulas PL

Ferraments

- **Prolog**
  - SWI-Prolog
  - SWI-Prolog (online)
- **Data Mining**
  - Python
  - Weka
  - RapidMiner
  - Orange
  - R
- **Apoio**
  - ChatGPT / CURSOS / CoPilot / Gemini – Atenção ao código que é gerado devem seguir as indicações e o modelos das aulas
- **Outras**
  - Se perceberem o código / programa, sabem como funciona, o que fazer e se sabem alterar

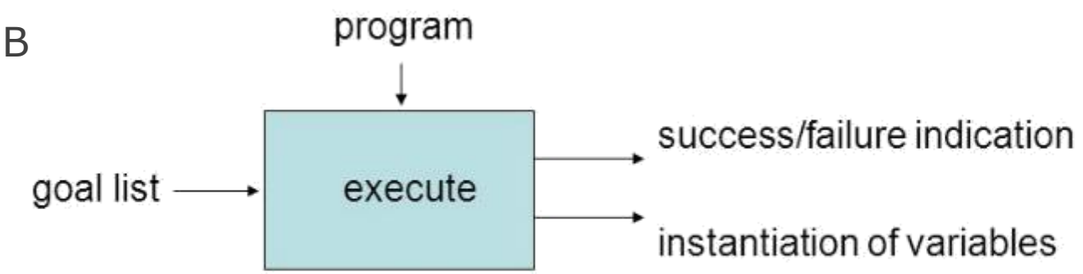
Prolog

- Linguagem de programação de alto nível baseada em **lógica** formal.
- Ao contrário das linguagens de programação tradicionais, baseadas na execução de **sequências** de **comandos**, o Prolog baseia-se em definir e resolver formulações **lógicas**.
- Normalmente designa-se o Prolog como uma linguagem declarativa ou linguagem baseada em regras porque os seus programas consistem numa lista de factos e regras.
- É utilizada extensivamente em aplicações de **inteligência artificial**, particularmente em sistemas periciais.
- A abreviatura **Prolog** significa Programming Logic.

***CFP**: O Prolog não deve ser visto como uma linguagem de programação, mas sim, como uma linguagem estrutural que representa o pensamento e o modela o conhecimento*

Prolog

- A **Semântica Declarativa** define se uma meta é verdadeira ou não em relação a um determinado programa e, se for verdadeira, para qual instanciação de variáveis é verdadeira.
- A **Semântica Processual** consiste em satisfazer uma lista de objetivos no contexto de um determinado programa. Gera a veracidade ou falsidade da lista de objetivos e as instanciações correspondentes das variáveis e retrocede automaticamente para verificar o seu substituto.
- **Factos**:
  - capital(Paris, France)
  - %teaches(A, B): person A teaches in course B
  - teaches(Sudhir,course001).
  - teaches(Surbhi,course002).



- Saber Mais

Prolog

- Em prolog fazemos **consultas** (*queries)* ao sistema que vai responder de acordo com os **factos** e **regras** criadas.
  - Factos
    - Permitem estabelecer um relacionamento existente entre objetos de um determinado contexto de discurso (p. ex. árvore genealógica)
      - pai(adão,cain).
      - pai(adão,abel).
      - pai(adão,seth).
      - pai(seth,enos).
  - Regras
    - Permitem definir novas relações em termos de outras relações já existentes. Por exemplo, a regra
      - avô(X,Y) :- pai(X,Z), pai(Z,Y).

Prolog

- Em prolog fazemos **consultas** (*queries)* ao sistema que vai responder de acordo com os **factos** e **regras** criadas.
  - <u>Consultas</u>
    - Permitem recuperar informações de um programa lógico.
    - Uma consulta pergunta se uma determinado relacionamento existe entre objetos.
      - ?- pai(adão,cain).
        - TRUE
      - ?- pai(adão,X).
        - X = cain ;
        - X = abel ;
        - X = seth ;
        - no
      - ?- avô(X,enos).
        - X = adão

      - pai(adão,cain).
      - pai(adão,abel).
      - pai(adão,seth).
      - pai(seth,enos).

Prolog – Operadores

- O operador **=** tenta unificar apenas o conteúdo
  - ?- X = 1 + 2.
    - X = 1 + 2
- O operador **is** força a avaliação aritmética
  - ?- X is 1 + 2.
    - X = 3
  - Se a variável à esquerda do operador **is** já estiver instanciada, o Prolog apenas compara o valor da variável com o resultado da expressão à direita de is
  - ?- X = 3, X is 1 + 2.
    - X = 3
  - ?- X = 5, X is 1 + 2.
    - no

Operador	Significado
X = Y	Igual a
X \= Y	Não igual a
X < Y	Menor que
Y > X	Maior que
Y =< X	Menor ou igual a
Y >= X	Maior ou igual a

Prolog – Alfabeto

- **Variáveis**: começa com uma letra maiúscula ou um sublinhado (_), seguido por qualquer número de letras, dígitos ou sublinhados.
- **Átomos**: sequências de caracteres alfanuméricos ou símbolos especiais que não começam com letra maiúscula ou sublinhado.
  - Os átomos são usados para representar constantes e nomes de predicados.
- **Números**: podem ser inteiros ou reais.
- **Símbolos especiais**: Prolog usa vários símbolos especiais para representar operadores, conectores lógicos e outros elementos da linguagem.
  - Por exemplo, a vírgula (",") é usada para separar itens em uma lista, e o ponto final (".") é usado para indicar o fim de uma regra ou consulta.
- **Termos compostos**: consistem em um átomo seguido por um conjunto de argumentos entre parênteses.
  - Os argumentos são separados por vírgulas e podem ser variáveis, átomos ou outros termos compostos.
  - Os termos compostos são usados para representar estruturas de dados mais complexas, como listas e árvores.

Prolog – Alfabeto

- Alfabeto do prolog

Alphabet of Prolog

The alphabet of PROLOG consists of:

- C — a set of constant symbols (or constants, for short),
- V — a set of variable symbols (or variables, for short),
- F — a set of function (term) symbols,
- P — a set of relation (predicate) symbols.

Meaning and Notation of Symbols

- Constants denote specific objects, items, elements, values, phenomena, etc. Constant names start with lower-case letters. Integers, rational numbers and strings are allowed (e.g. 'A small cat').
- Variables are used to denote the same elements in case the precise name of an element is currently not known, unimportant, or a class of elements is to be represented. Variable names start with an upper-case letter.
- Functional symbols serve as complex object constructors. Such objects have a root symbol (an element of F) and a number of arguments. They follow the tree-like structure.
- Predicate symbols are used to define facts (relations). A fact can be *true* or *false*.

Logical connectives

- `:-` is equivalent of implication (**if**),
- `,` is equivalent of conjunction (**and**)n,
- `;` is equivalent of disjunction (**or**).

Facts

```
pred(arg1, arg2, ... argN).
```

Clauses

```
h :- p1, p2, ..., pk.
h :- q1, q2, ..., qm.
```

Clauses — disjunction

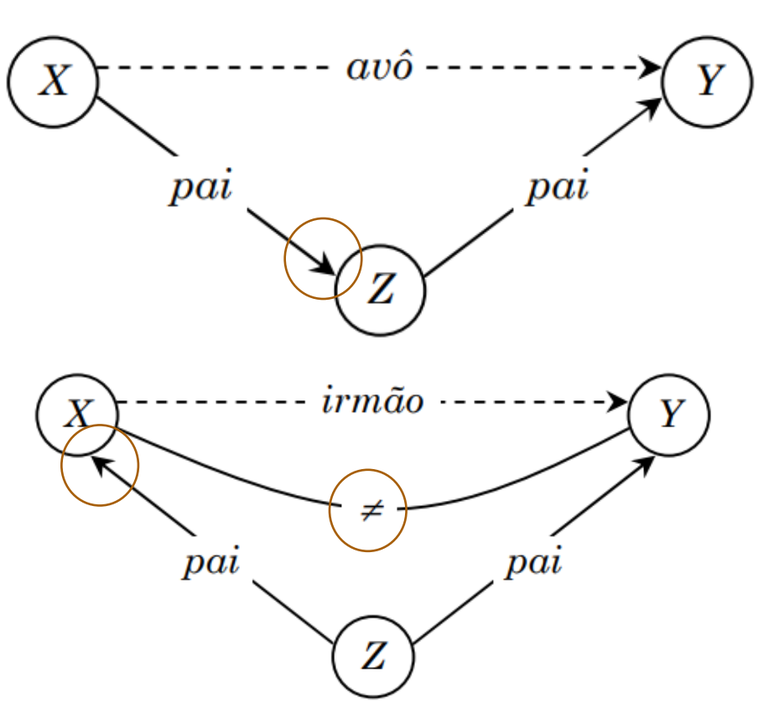
```
h :- p1, p2, ..., pk; q1, q2, ..., qm.
```

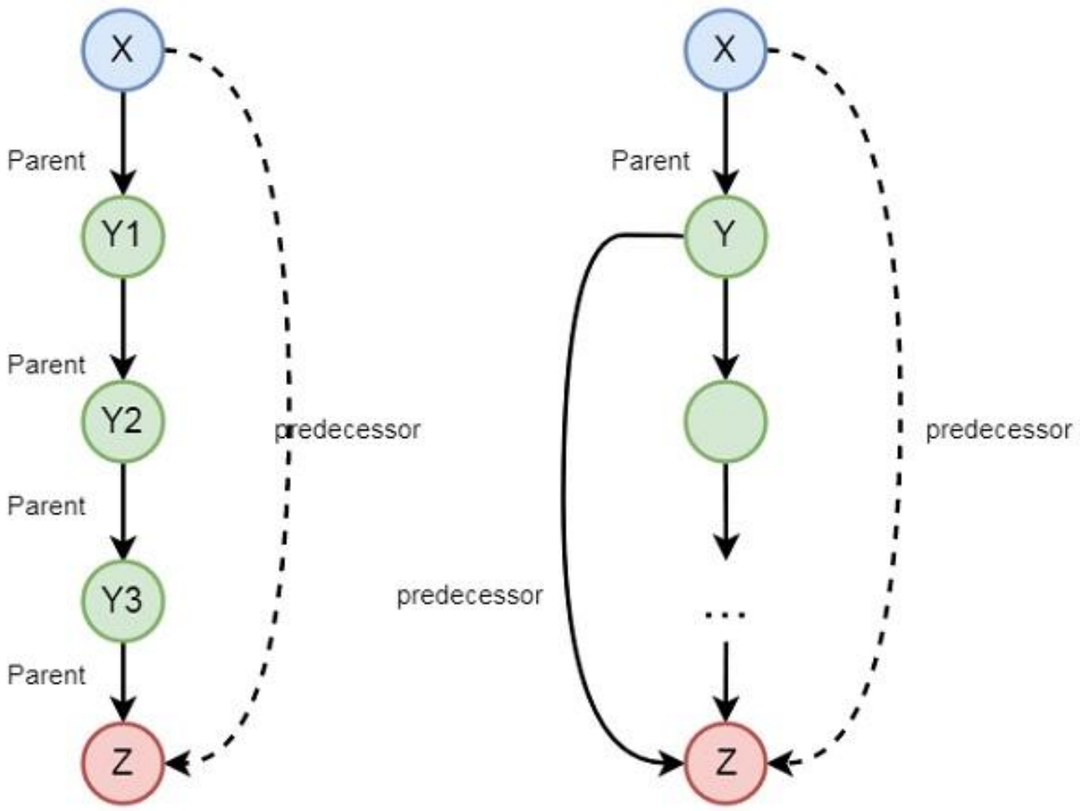
Prolog – Variáveis

- São cadeias de letras, dígitos e caracteres '_', sempre começando com letra **maiúscula** ou com o caractere **'_'**
  - Aconselha-se a utilização sempre de variáveis maiúsculas.
- O ambiente de uma variável está limitado à mesma regra ou dentro de uma pergunta.
- Variável **anónima** (não interessa)
  - Quando uma variável aparece em uma única cláusula, não é necessário utilizar um nome para ela
  - Por exemplo, temfilho(X) :- progenitor(X,Y).
  - Apenas pretende indicar se tem ou não filho, não "interessa" o nome, assim deve ser utilizado uma variável anónima
    - temfilho(X) :- progenitor(X,_).

Prolog – Relacionamentos

- Grafos de **relacionamentos**

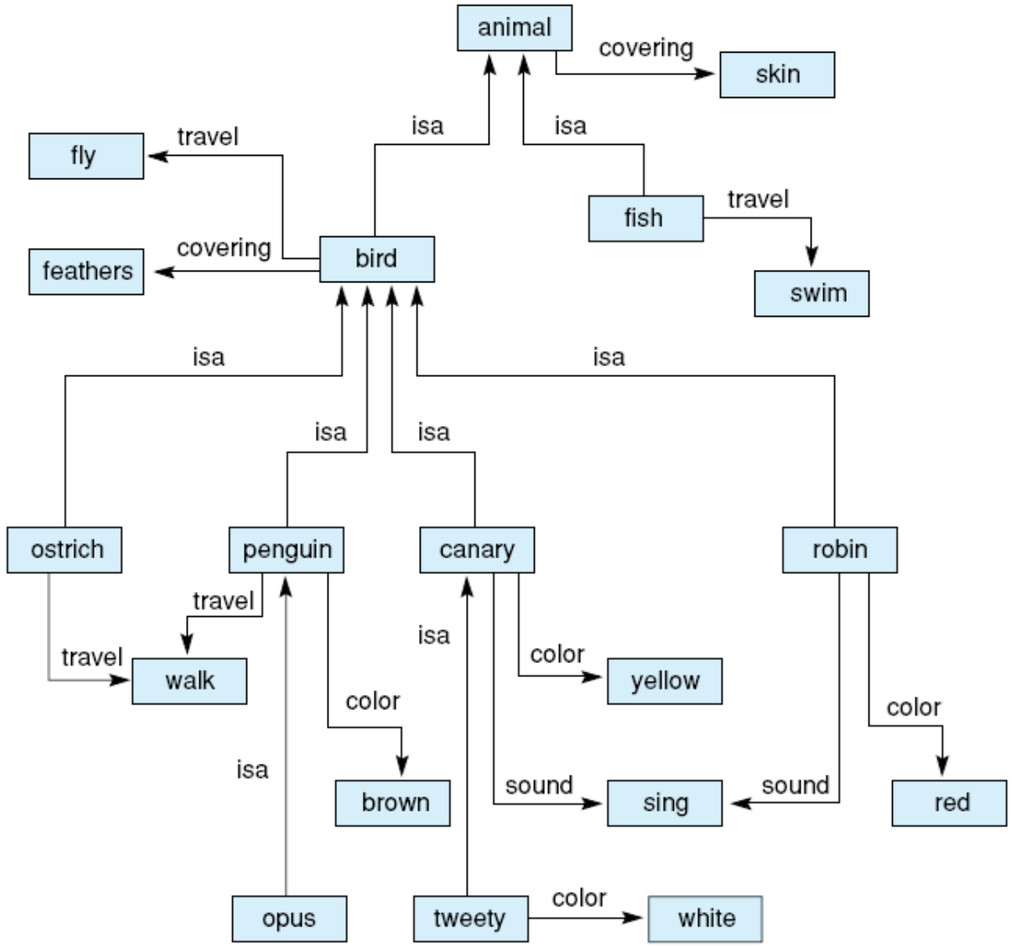




- Em Prolog, essa regra é escrita como:
  - irmão(X,Y) :- pai(Z,X), pai(Z,Y), X\=Y.

Prolog – Relacionamentos

- Grafos de **relacionamentos**




```
%FACTS

%Bird %animal's childs
isa(bird, animal).
isa(fish, animal).

%bird's childs
isa(ostrich, bird).
isa(penguin, bird).
isa(canary, bird).
isa(robin, bird).

%penguin's childs
isa(opus, penguin).

%canary's childs
isa(tweety, canary).

%animal's property
hasprop(animal, covering, skin).

%bird's property
hasprop(bird, travel, fly).
hasprop(bird, covering, feathers).

%fish's property
hasprop(fish, travel, swim).
```

```
%ostrich's property
hasprop(ostrich, travel, walk).

%penguin's property
hasprop(penguin, travel, walk).
hasprop(penguin, color, brown).

%canary's property
hasprop(canary, color, yellow).
hasprop(canary, sound, sing).

%robin's property
hasprop(robin, color, red).
hasprop(robin, sound, sing).

%tweety's property
hasprop(tweety, color, white).

%RULES
hasproperty(Object, Property, Value) :-
    hasprop(Object, Property, Value).

hasproperty(Object, Property, Value) :-
   isa(Object, Parent),
   hasproperty(Parent, Property, Value),
   \+ hasprop(Object, Property, _).
% Let object property override
```

Prolog – Exemplos

no space

Arguments/Objects

likes(john, jane). ← dot necessary to end statements

Facts → likes(jane, john).

likes(jack, jane).

Predicate name

variables

Rule → friends(X, Y) :- likes(X, Y), likes(Y, X).

head

body

```
grandmother(X, Y) :- mother(X, Z), parent(Z, Y).
parent(X, Y) :- mother(X, Y).
parent(X, Y) :- father(X, Y).

mother(mary, stan).
mother(gwen, alice).
mother(valery, gwen).
father(stan, alice).
```

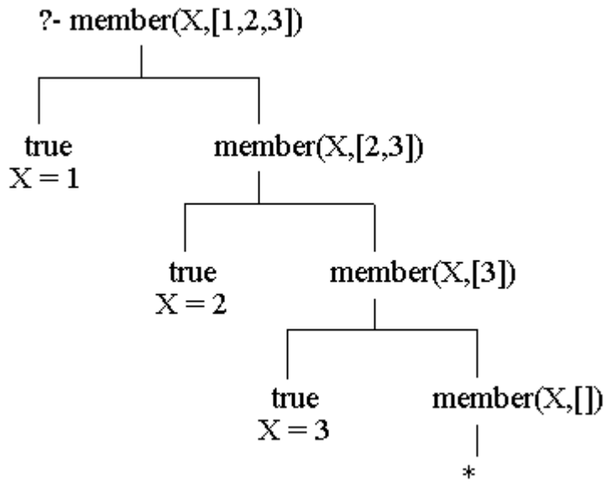
?- likes(john, jane). ← dot necessary

true. ← answer from prolog interpreter

sign on prolog query prompt

variables

?- friends(X, Y).



Prolog – Exemplos

Program

```
elephant(fred).
elephant(mary).
lion(joe).
tiger(alice).
animal(fred) :- elephant(fred).
animal(mary) :- elephant(mary).
animal(joe) :- lion(joe).
animal(alice) :- tiger(alice).
alive(X) :- animal(X).
```

```
alive(X).
X = fred
X = mary
X = joe
X = alive
?-
```

Seja a base de dados:

```
grande(urso).                    % Cláusula 1
grande(elefante).                % Cláusula 2
pequeno(gato).                   % Cláusula 3
marrom(urso).                    % Cláusula 4
preto(gato)                      % Cláusula 5
cinza(elefante).                 % Cláusula 6
escuro(Z) :-                     % Cláusula 7
    preto(Z).
escuro(Z) :-                     % Cláusula 8
    marrom(Z).
```

Qual o resultado da consulta:

- **?- escuro(X), grande(X).**

Prolog – Exemplos

```
f(a).
f(b).
g(a).
g(b).
h(b).
k(X):- f(X), g(X), h(X).
```

Base de dados

```
?- k(Y).
```

Consulta

Prolog

Documentação de suporte

- **Drive**
  - Livros, Slides Tutorial
- **Complementos**
  - Tutorial
  - Exemplos

Exercícios

Exercício 1

- A Rita supervisiona o João e o Rui, na empresa de consultadoria C. O Rui exerce funções também na empresa S, de software, onde supervisiona o Miguel. O João orienta uma estagiária, a Iva. A Rita é chefe da empresa C, enquanto que o Vítor chefia a empresa S, supervisionando o Rui e o Tomás. Por sua vez, o Tomás orienta o estagiário Ivo.
  a) Utilizando o predicado **trabalhapara**(X,Y,E) (ou seja, X trabalha sob a direção de Y na empresa E), represente em Prolog todas relações de chefia/orientação.
  b) Represente o facto **chefe**(X,E), onde X representa o chefe da empresa E.
  c) Codifique a regra **mesmonivel**(X,Y), que é verdade se X e Y trabalham para o mesmo chefe/supervisor, sendo que a empresa de X e Y não interessa (pode ser igual ou diferente).
  d) Codifique a regra **empregado**(X,E), onde X trabalha na empresa E (nota: trabalhadores e chefes são empregados).
  e) Implemente a regra **temchefe**(X,Y), que é verdade se X trabalhar na empresa E onde Y é o chefe.
  f) Codifique a regra **acima**(X,Y,E), onde Y supervisiona X ou supervisiona alguém que supervisiona X na empresa E.
  g) Crie um facto que permita dar um **sexo** (m ou f) a cada um dos empregados. Utilizando este facto, defina a regra **perigoassedio**(X,Y) que é X e Y forem de sexos diferentes e se um trabalhar para o outro.
  h) Formule em Prolog as seguintes questões (perguntas simples, sem utilizar o findall):
     1. O João trabalha para a Rita?
     2. Quem são os subordinados da Rita (quem ela supervisiona)?
     3. Quem é do mesmo nível do João? e do Rui?
     4. Quem trabalha acima do Ivo?
     5. Quais são todos os trabalhadores da empresa cuja chefe é a Rita?
     6. Quem trabalha para duas empresas diferentes?
     7. Com quem existe perigo de assédio com o João?
     8. Quem trabalha só para uma única empresa?